

ANALIZA I INTEGRACIJA KREACIJSKIH DIZAJNERSKIH PATERNA

Uvod i kontekst postojećeg sistema

Sistem modelira ključne domenske entitete kao što su `Korisnik`, `Rezervacija`, `ParkingObjekat`, `Transakcija` i `Izvjestaj`. Također, sistem koristi *Strukturne paterne* (`ParkingReservationFacade` koji pojednostavljuje interakciju sa podsistemima i `PaymentInterface` sa adapterima `StripeAdapter` i `PayPalAdapter` koji izoluju eksterne servise za plaćanje). Trenutna struktura se oslanja na direktno vezivanje (*tight coupling*) i prebacivanja prevelike odgovornosti na klijentske klase, što narušava **Single Responsibility Principle (SRP)** i **Open/Closed Principle (OCP)**.

Potrebe za kreacijskim paternima

Kreacijski dizajnerski paterni (*Creational Design Patterns*) bave se mehanizmima kreiranja objekata, stvarajući objekte na način koji je prilagođen situaciji. Bez njihove primjene, sistem se suočava sa sljedećim problemima:

- **Direktno instanciranje (Ključna riječ `new`):** Kada klasa `Korisnik` ili fasada direktno pozivaju konstruktore drugih klasa, one postaju zavisne o konkretnim implementacijama, a ne apstrakcijama.
- **Kompleksnost inicijalizacije:** Entiteti poput `Izvjestaj` ili `Rezervacija` zahtijevaju postavljanje većeg broja parametara (npr. `periodOd`, `periodDo`, `format`, specifični fajl podaci). Ako se logika validacije i sklapanja nalazi unutar klijentske klase, kod postaje težak za održavanje i podložan greškama.
- **Nemogućnost proširenja:** Ako se u sistem uvede novi tip izvještaja ili novi tip korisnika, moraju se mijenjati postojeće klase koje ih kreiraju, što direktno krši OCP.

Odabir kreacijskih paterna

Patern 1: Factory Method (Fabrička metoda)

- **Lokacija problema na osnovnom dijagramu:** Klasa `Korisnik` trenutno sadrži metodu sa stereotipom `<<Create>>` `Korisnik(...)` i direktno upravlja kreiranjem instanci. Također, sistem generiše različite vrste izvještaja (`Izvjestaj`) koji zavise od enumeracije `FormatIzvjestaja` (`PDF`, `EXCEL`, `WORD`).
- **Rješenje i opravdanje:** Uvođenjem **Factory Method** paterna za kreiranje izvještaja, izolujemo logiku instanciranja. Umjesto da klijent (npr. administrator ili fasada) direktno instancira konkretni izvještaj i ručno postavlja attribute, kreira se apstraktna klasa (ili interfejs) `IzvjestajFactory` sa metodom `kreirajIzvjestaj()`, koju nasljeđuju konkretne fabrike (`PDFIzvjestajFactory`,

ExcelIzvjestajFactory). Ovo omogućava dodavanje novih formata izvještaja u budućnosti bez izmjene ijedne linije postojećeg klijentskog koda.

Patern 2: Builder (Graditelj)

- **Lokacija problema na osnovnom dijagramu:** Klasa Rezervacija je centralni i najkompleksniji entitet sistema. Sadrži mnoštvo atributa: rezervacijaID, vrijemePocetka, vrijemeIsteka, QRCode, status, vlasnikID i parkingID. Kreiranje rezervacije zahtijeva korak-po-korak provjeru, generisanje QR koda, kalkulaciju vremena i vezivanje za korisnika i parking objekat.
- **Rješenje i opravdanje:** Patern **Builder** razdvaja konstrukciju složenog objekta od njegove reprezentacije. Uvođenjem klase RezervacijaBuilder, omogućavamo kreiranje objekta Rezervacija kroz jasan i čitljiv lanac metoda (*fluent interface*), npr. `setKorisnik().setParking().setVrijeme().build()`. Fasada `ParkingReservationFacade` tada koristi ovaj Builder kako bi bezbjedno i validirano sklopila objekat rezervacije, smanjujući kompleksnost konstruktora i spriječavajući slanje pogrešnih parametara.